sdf-xarray: Modernised interactivity

Joel Adams, Peter Hill, Liam Pattinson, Shaun Doherty, Sviatoslav Shekhanov, Lawrence Dior, Chris Herdman

PlasmaFAIR

EPSRC Grant

Motivation

- EPOCH was written before the standardisation of output file types
- Leading to a custom binary file type being used
- Self Describing Format (SDF) was born
- As a result, several wrappers were created for it including...

IDL® Software

Paid closed-source software :(

GDL

Rather old but open-source IDL

open PMD

Not widely adopted

MathWorks®

Paid closed-source software :(

sdf_helper

Custom Python reader

Introducing...

What is sdf-xarray?

- Another open-source Python package… yeah, I know, bare with me!
- Initially developed in 2023 by Peter Hill under the {PlasmaFAIR}
- Uses the sdf.c library to translate files to xarray
- But wait, I hear you ask. What is xarray?!

What is xarray?

> “Xarray introduces labels in the form of dimensions, coordinates and attributes on top of raw NumPy-like multidimensional arrays, which allows for a more intuitive, more concise, and less error-prone developer experience.” - https://xarray.dev

- Open-source
- Follows a similar data model to the very popular NetCDF file format (not quite, almost industry standard)
- Interoperable with the scientific Python ecosystem including NumPy, Dask, Pandas, and Matplotlib.
- Allows conversion from SDF to other popular file formats such as Zarr, NetCDF, and HDF5

Why use sdf-xarray

- **Data conversion** - Data can be easily converted to NumPy or Pandas if you prefer to work with raw data
- **Data labelling** - All SDF files come with a bunch of information about them such as the dimensions, units and name of the variable. These are added to each variable making it much easier to read
- **Lazy/partial loading** – We can reduce the RAM requirements when loading large SDF files as we only load the actual array values when they are needed thanks to dask
- **Plotting** - You can easily plot variables across several dimensions including time using built in matplotlib support
- **Animating** - Visualise your data across time or slice through another dimension and make beautiful GIFs for your presentations
- **Deck loading** - automatically loads in the associated input.deck utilised when creating the simulation so that you can access setup variables using epydeck
- **Unit conversion** - Convert dimensions and variable units to other types with the help of pint

Documentation, Issues, Support and Citations

- We aren't magicians and can't tell if you're using the package...
- So please tell us if you are using it!
  - Support us by starring the repository on https://github.com/epochpic/sdf-xarray
  - Cite Zenodo 10.5281/zenodo.15351323 (JOSS paper to come) when publishing
- Having a problem with it?
  - Check the "known issues" page on the documentation
  - Raise an issue on GitHub
  - If you can't because of company policy email me instead
- New Feature requests?
  - Create a new issue with the suggestion
- Documentation is available at https://sdf-xarray.readthedocs.io

Interfacing with sdf-xarray

xr-tui

```
>>> import sdf_xarray as sdfxr
>>> sdfxr.open_mfdataset("/Users/joel/Source/sdf-xarray/docs/epoch_workshop_202\
6/datasets/3_5_Gaussian_beam")
<xarray.Dataset> Size: 84MB
Dimensions:                  (time: 11, X_Grid_mid: 800, Y_Grid_mid: 400,
                              X_Grid: 801, Y_Grid: 401)
Coordinates:
  * time                     (time) float64 88B 2.801e-17 1.003e-14 ... 1e-13
  * X_Grid_mid               (X_Grid_mid) float64 6kB 1.25e-08 ... 1.999e-05
  * Y_Grid_mid               (Y_Grid_mid) float64 3kB -4.987e-06 ... 4.987e-06
  * X_Grid                   (X_Grid) float64 6kB 0.0 2.5e-08 ... 2e-05
  * Y_Grid                   (Y_Grid) float64 3kB -5e-06 -4.975e-06 ... 5e-06
Data variables:
    Wall_time                (time) float64 88B 0.002469 0.2344 ... 2.28 2.511
    Derived_Poynting_Flux_x  (time, X_Grid_mid, Y_Grid_mid) float64 28MB dask.array<chu
    Derived_Poynting_Flux_y  (time, X_Grid_mid, Y_Grid_mid) float64 28MB dask.array<chu
    Derived_Poynting_Flux_z  (time, X_Grid_mid, Y_Grid_mid) float64 28MB dask.array<chu
Attributes: (12/22)
    filename:         /Users/joel/Source/sdf-xarray/docs/epoch_workshop_2026/...
    file_version:     1
    file_revision:    4
    code_name:        Epoch2d
    step:             0
    time:             2.80090202489025e-17
    ...               ...
    compile_flags:    unknown
    defines:          58753092
    compile_date:     Thu Feb 26 09:55:50 2026
    run_date:         Fri Apr 10 10:42:21 2026
    io_date:          Fri Apr 10 10:42:21 2026
    deck:             {'control': {'nx': 800, 'ny': 400, 't_end': 1e-13, 'x_m...
```

```
sdfxr.open_dataset(dataset_path_2d / "0000.sdf")
```

xarray.Dataset

▶ Dimensions: (**X_Grid_mid**: 100, **Y_Grid_mid**: 100, **X_Grid**: 101, **Y_Grid**: 101)

▼ Coordinates:

X_Grid_mid	(X_Grid_mid)	float64	-4.9e-06 -4.7e-06 ... 1.49e-05
Y_Grid_mid	(Y_Grid_mid)	float64	-9.9e-06 -9.7e-06 ... 9.9e-06
X_Grid	(X_Grid)	float64	-5e-06 -4.8e-06 ... 1.5e-05
Y_Grid	(Y_Grid)	float64	-1e-05 -9.8e-06 ... 9.8e-06 1e-05

▼ Data variables:

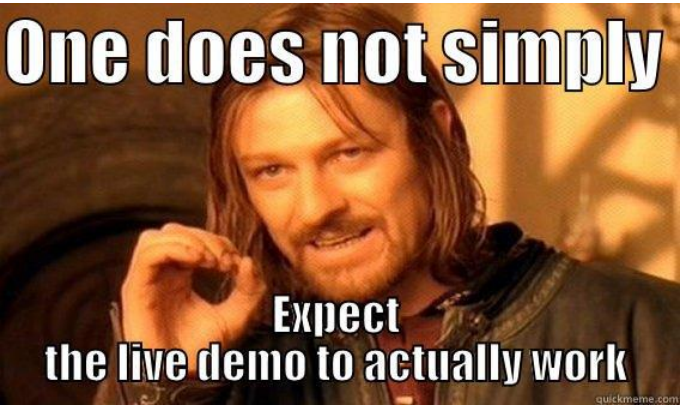
Wall_time	()	float64	...
Electric_Field_Ey	(X_Grid_mid, Y_Grid_mid)	float32	...
Derived_Number...	(X_Grid_mid, Y_Grid_mid)	float32	...
Derived_Number...	(X_Grid_mid, Y_Grid_mid)	float32	...
Derived_Poyntin...	(X_Grid_mid, Y_Grid_mid)	float64	...
Derived_Poyntin...	(X_Grid_mid, Y_Grid_mid)	float64	...
Derived_Poyntin...	(X_Grid_mid, Y_Grid_mid)	float64	...

▶ Attributes: (22)

```
xr-tui
▼ xarray Dataset: /Users/joel/Source/sdf-xarray/docs/epoch_workshop_2026/datasets/3_5_Gaussian_beam/*.sdf
├── ▼ File Information
│   ├── File Glob: *.sdf
│   ├── Total Files Loaded: 11
│   ├── Total Files Size: 80.70 MB
│   ├── First File Created: Thu Apr 16 15:13:40 2026
│   ├── Last File Created: Thu Apr 16 15:13:40 2026
│   └── ▶ Individual File Information
└── ▼ Root (Data Variables: 1 Coordinates: 1)
    ├── ▶ Attributes (22)
    ├── ▼ Dimensions
    │   └── time: 11
    ├── ▼ Coordinates
    │   └── ▶ time: ('time',) float64 88.00 B
    ├── ▼ Data Variables
    │   └── ▶ Wall-time: ('time',) float64 88.00 B
    └── ▼ Group: Derived (Data Variables: 0 Coordinates: 1)
        ├── ▼ Dimensions
        │   └── time: 11
        ├── ▼ Coordinates
        │   └── ▶ time: ('time',) float64 88.00 B
        ├── ▼ Data Variables
        └── ▼ Group: Poynting_Flux (Data Variables: 3 Coordinates: 3)
            ├── ▼ Dimensions
            │   ├── time: 11
            │   ├── X_Grid_mid: 800
            │   └── Y_Grid_mid: 400
            ├── ▼ Coordinates
            │   ├── ▶ time: ('time',) float64 88.00 B
            │   ├── ▶ X_Grid_mid: ('X_Grid_mid',) float64 6.25 KB
            │   └── ▶ Y_Grid_mid: ('Y_Grid_mid',) float64 3.12 KB
            └── ▼ Data Variables
                ├── ▶ x: ('time', 'X_Grid_mid', 'Y_Grid_mid') float64 26.86 MB
                ├── ▶ y: ('time', 'X_Grid_mid', 'Y_Grid_mid') float64 26.86 MB
```

Broad EPOCH Analysis Modules

sdf-xarray: Parse SDF files
epydeck: Parsing/create input.deck files into Python dictionaries
epyscan: Sample simulation parameter space


One does not simply
Expect
the live demo to actually work

Me: watch my live demo
My computer:
"You fell victim to one of the classic blunders"

TIME FOR A LIVE DEMO
WHAT COULD POSSIBLY GO WRONG?

Recommended setup

Visual Studio Code + Jupyter Extension

```
> uv venv
> # activate venv
> uv pip install "sdf-xarray[jupyter]" matplotlib pooch
```

Download the Jupyter notebook from the “Workshop Details” page in the “EPOCH Workshop 2026” section on https://sdf-xarray.readthedocs.io